

3.4 – Remote Development Workflow

Writing Code on Your Laptop, Running It on the Pi

EdgeAI Engineering Bootcamp — Analog Data

What You Will Learn

- Edit Pi files directly from VS Code on your laptop using Remote-SSH
 - Sync files from your laptop to the Pi fast using `rsync`
 - Keep terminal sessions alive even after SSH disconnects using `tmux`
-
-

Part 1 — The Problem with SSHing into the Pi

The typical student workflow looks like this:

1. Write code in a nice editor on the laptop
2. Copy the file to the Pi (with `scp` or a USB drive)
3. SSH into the Pi, run the file, see an error
4. Go back to the laptop, fix the code
5. Copy again to the Pi
6. SSH again, run again

This back-and-forth is slow and annoying. The tools in this section eliminate steps 2, 3, 4, 5, and 6 — you edit directly on the Pi from your laptop with full IDE features, and your files sync instantly.

Part 2 — VS Code Remote-SSH

What It Is

VS Code has an extension called **Remote-SSH** that lets you open any folder on a remote machine (like your Pi) directly in VS Code on your laptop. You get:

- Full IntelliSense (autocomplete, type hints, error highlighting)
- Integrated terminal connected directly to the Pi
- File explorer showing the Pi's filesystem
- Debugger running on the Pi

It looks and feels exactly like you are editing a local project — but every file is actually on the Pi.

Step 1 — Install the Extension

On your laptop, open VS Code. Press `Ctrl + Shift + X` to open Extensions. Search for `Remote - SSH` and install it (by Microsoft).

Step 2 — Set Up SSH Key Authentication (Recommended)

Typing a password every time you connect is slow. SSH keys let you connect without a password.

On your laptop terminal:

```
# Generate an SSH key pair (press Enter for all prompts to use defaults)
ssh-keygen -t ed25519 -C "my-pi-key"

# Copy the public key to your Pi
ssh-copy-id pi@192.168.1.x
```

Test it:

```
ssh pi@192.168.1.x
# Should connect without asking for password
```

Step 3 — Add Your Pi to SSH Config

On your laptop, edit (or create) `~/.ssh/config` :

```
Host raspberrypi
  HostName 192.168.1.x      ← replace with your Pi's IP address
  User pi
  IdentityFile ~/.ssh/id_ed25519
```

Now instead of typing `ssh pi@192.168.1.x` you just type `ssh raspberrypi`. VS Code uses this config too.

Step 4 — Connect VS Code to the Pi

1. Press `Ctrl + Shift + P` to open the command palette
2. Type `Remote-SSH: Connect to Host`
3. Select `raspberrypi` (from your SSH config)
4. VS Code opens a new window connected to the Pi
5. Click `Open Folder` and navigate to your project folder (e.g. `/home/pi/edgeai`)

You are now editing files directly on the Pi in VS Code with full IntelliSense. The integrated terminal at the bottom is a live SSH session into the Pi.

Finding Your Pi's IP Address

On the Pi terminal:

```
hostname -I
# 192.168.1.45 - use this in your SSH config
```

Or from your laptop:

```
ping raspberrypi.local
# Shows the IP address of the Pi
```

Part 3 — `rsync` — Fast File Synchronisation

What It Is

`rsync` is a command-line tool that copies files from one location to another, but it is far smarter than `cp` or `scp`. It:

- Only transfers files that have **changed** since the last sync
- Transfers only the **changed parts** of large files (not the whole file)
- Preserves file permissions, timestamps, and symlinks
- Shows progress so you can see what is happening

For copying a project folder with 200 Python files where only 2 changed — `rsync` transfers 2 files. `scp` transfers all 200.

Basic Syntax

```
rsync -avz --progress source/ destination/
```

Flag	Meaning
<code>-a</code>	Archive mode — preserves permissions, timestamps, symlinks. Equivalent to <code>-rlptgoD</code> . Always use this.
<code>-v</code>	Verbose — shows each file being transferred
<code>-z</code>	Compress data during transfer — faster on slow WiFi
<code>--progress</code>	Show transfer speed and percentage for each file

Syncing from Laptop to Pi

```
# Sync your local project folder to the Pi
rsync -avz --progress ./edgeai/ pi@192.168.1.x:/home/pi/edgeai/

# Using your SSH config alias
rsync -avz --progress ./edgeai/ raspberrypi:/home/pi/edgeai/
```

The trailing slash on the source matters.

`./edgeai/` (with slash) → sync the **contents** of the folder

`./edgeai` (without slash) → sync the folder **itself**, creating `/home/pi/edgeai/edgeai/`

Almost always use the trailing slash on the source.

Excluding files you don't want to sync

```
rsync -avz --progress \
  --exclude '.git/' \
  --exclude '__pycache__/' \
  --exclude '*.pyc' \
  --exclude '.env' \
  --exclude 'venv/' \
  ./edgeai/ raspberrypi:/home/pi/edgeai/
```

Or use an exclude file:

```
# Create a file listing what to exclude
cat > .rsyncignore << EOF
.git/
__pycache__/
*.pyc
.env
venv/
*.log
EOF

# Use it with --exclude-from
rsync -avz --progress --exclude-from='.rsyncignore' ./edgeai/
raspberrypi:/home/pi/edgeai/
```

Syncing from Pi back to Laptop (pulling files)

```
# Pull logs or output files back to your laptop
rsync -avz --progress raspberrypi:/home/pi/edgeai/logs/ ./logs/
```

Dry run — Preview what would be transferred

```
rsync -avz --progress --dry-run ./edgeai/ raspberrypi:/home/pi/edgeai/
```

`--dry-run` shows exactly what `rsync` would do without actually transferring anything. Always use this first when syncing to a new destination to make sure you are not accidentally overwriting something.

Part 4 — `tmux` — Persistent Terminal Sessions

The Problem It Solves

You SSH into your Pi, start a long-running process (training a model, running inference, watching logs), and then your WiFi drops for 10 seconds. SSH disconnects — and your process is killed. You have to start everything again.

`tmux` keeps your terminal session alive on the Pi **even after the SSH connection drops**. When you reconnect, you re-attach to the same session and everything is exactly where you left it.

Installing `tmux`

On the Pi:

```
sudo apt install tmux -y
```

Core Concept — Sessions, Windows, and Panes

```
tmux session
├─ window 1 (e.g. "inference")
│   ├── pane 1 - python3 inference.py
│   └─ pane 2 - journalctl -f
└─ window 2 (e.g. "logs")
    └─ pane 1 - tail -f system.log
```

A **session** is the top-level container that survives disconnects. A **window** is like a tab inside the session. A **pane** is a split within a window.

Essential `tmux` Commands

All `tmux` keyboard shortcuts start with the **prefix key**: `Ctrl + B`. Press `Ctrl + B`, release, then press the next key.

Starting and attaching to sessions

```
# Start a new named session
tmux new -s edgeai

# Detach from current session (session keeps running on Pi)
# Ctrl + B, then D

# List all active sessions
tmux ls

# Re-attach to a session by name
tmux attach -t edgeai

# Re-attach to the most recent session
tmux attach
```

Windows (tabs)

Shortcut	Action
Ctrl+B c	Create a new window
Ctrl+B ,	Rename current window
Ctrl+B n	Next window
Ctrl+B p	Previous window
Ctrl+B 0-9	Switch to window by number
Ctrl+B w	List all windows and pick one

Panes (splits)

Shortcut	Action
Ctrl+B %	Split pane vertically (left/right)
Ctrl+B "	Split pane horizontally (top/bottom)
Ctrl+B ←→↑↓	Move between panes
Ctrl+B z	Zoom current pane to full screen (toggle)
Ctrl+B x	Close current pane

Scrolling through output

Shortcut	Action
Ctrl+B [	Enter scroll mode (use arrow keys to scroll)
q	Exit scroll mode

Typical tmux Workflow for Pi Development

```
# 1. SSH into Pi
ssh raspberrypi

# 2. Start a new tmux session named 'work'
tmux new -s work

# 3. In the first window, start your inference service
python3 /home/pi/edgeai/inference.py

# 4. Create a second window for logs
# Ctrl+B c
journalctl -u edgeai -f

# 5. Create a third window for file editing / commands
# Ctrl+B c
cd /home/pi/edgeai

# 6. When you need to leave – detach from the session
# Ctrl+B d
# Session is still running on the Pi. SSH disconnects safely.

# 7. Come back later and re-attach
ssh raspberrypi
tmux attach -t work
# Everything is exactly as you left it
```

Quick Reference Card

VS Code Remote-SSH:

Task	How
Connect to Pi	Ctrl+Shift+P → Remote-SSH: Connect to Host
Open a folder on Pi	File → Open Folder after connecting
Open integrated terminal	Ctrl+`

rsync:

Task	Command
Sync laptop → Pi	rsync -avz --progress ./project/ raspberrypi:/home/pi/project/
Sync Pi → laptop	rsync -avz --progress raspberrypi:/home/pi/project/logs/ ./logs/
Dry run (preview)	Add --dry-run

Task	Command
Exclude files	<code>--exclude '.git/' --exclude '__pycache__/'</code>

tmux:

Task	Command / Shortcut
New session	<code>tmux new -s name</code>
Detach	<code>Ctrl+B d</code>
Re-attach	<code>tmux attach -t name</code>
List sessions	<code>tmux ls</code>
New window	<code>Ctrl+B c</code>
Split horizontal	<code>Ctrl+B "</code>
Split vertical	<code>Ctrl+B %</code>
Switch panes	<code>Ctrl+B ←→↑↓</code>
Scroll mode	<code>Ctrl+B [</code>

Summary Checklist

- ☐ I have installed the VS Code Remote-SSH extension and connected to my Pi
- ☐ I have set up SSH key authentication so I connect without a password
- ☐ I can open a Pi project folder directly in VS Code
- ☐ I can use `rsync -avz --progress` to sync files from my laptop to the Pi
- ☐ I know what the trailing slash means in rsync and why it matters
- ☐ I can exclude files from rsync using `--exclude`
- ☐ I can use `--dry-run` to preview a sync before running it
- ☐ I can start a named `tmux` session and detach from it
- ☐ I can re-attach to an existing tmux session after reconnecting via SSH
- ☐ I can create multiple windows and panes inside a tmux session

Notes prepared by Analog Data — EdgeAI Engineering Bootcamp Copyright 2026 Analog Data. Strictly for enrolled students only.